

Java Programming Fundamentals



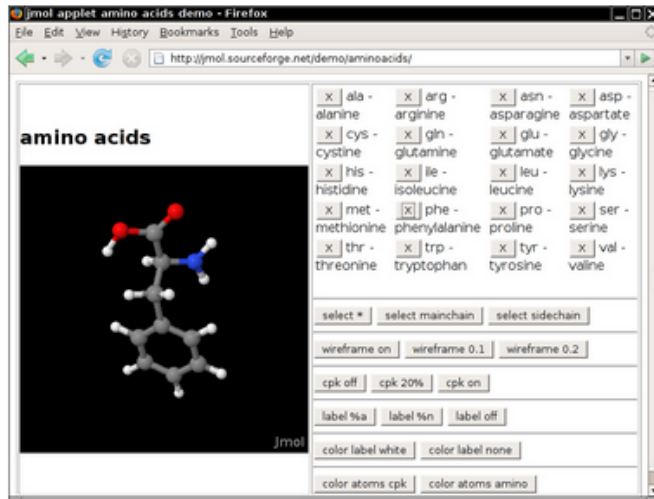
Java as a Programming Platform

- Java is a well-designed programming language.
 - But it isn't the epitome of elegance.
 - It's a “blue collar” language.
- Java is a platform:
 - Good programming language.
 - Efficient and secure execution environment.
 - Huge library.
- That combination has made it irresistible for over 20 years.

The “White Paper ” Buzzwords

1. Simple
2. Object-Oriented
3. Distributed
4. Robust
5. Secure
6. Architecture-neutral
7. Portable
8. Interpreted
9. High-performance
10. Multithreaded
11. Dynamic

Java Applets and the Internet



- Flash back to 1995.
- Browsers were new and served static web pages.
- Java applets made them *alive*.
- Applet code is downloaded from the internet.
- Runs safely in the “sandbox”.
- Unfortunately, the initial excitement turned into frustration.
 - Netscape and Internet Explorer used different Java versions.
 - Flash provided dynamic effects in browsers.
- Sun and Oracle were slow to react to security breaches.
- Nowadays, Java is used in servers and smart phones.

A Short History of Java

- 1991: James Gosling worked on “Project Green”, a system for consumer devices.
- He designed a programming language, originally called “Oak”.
- That name was trademarked, so it was renamed to “Java”.
- 1992: The first project was released, a TV switchbox called “*7”.
- Nobody cared, and the project was renamed “First Person, Inc.”
- 1994: Still nobody cared, and Gosling realized that they could build a “really cool browser...architecture-neutral, real-time, reliable, secure.”
- 1995: HotJava was released.
- 1996: Java 1.0 was released.
- 1998: Java grows up with Java 2 release. SE, ME, EE editions.
- 2014: Java 8 has major new language features.
- 2016: Java is again the #1 language in the TIOBE ratings.

Java Versions

Version	Year	New Language Features	Number of Classes and Interfaces
1.0	1996	The language itself	211
1.1	1997	Inner classes	477
1.2	1998	The strictfp modifier	1,524
1.3	2000	None	1,840
1.4	2002	Assertions	2,723
5.0	2004	Generic classes, “for each” loop, varargs, autoboxing, metadata, enumerations, static import	3,279
6	2006	None	3,793
7	2011	Switch with strings, diamond operator, binary literals, exception handling enhancements	4,024
8	2014	Lambda expressions, interfaces with default methods, stream and date/time libraries	4,240

Common Misconceptions About Java

Java is an extension of HTML or XML.

Java is a programming language.

Java is an easy programming language to learn.

No programming language as powerful as Java is easy.

Java will become a universal programming language for all platforms.

This is possible in theory, but today, Java is most commonly used in backend systems and Android applications.

Java is just another programming language.

Java has a support system that far exceeds that of most other languages.

Java is proprietary, and it should therefore be avoided.

Java is open source.

Common Misconceptions About Java

Java is interpreted, so it is too slow for serious applications.

The just-in-time compiler can produce code that is as fast as C++, and sometimes faster.

All Java programs run inside a web page.

Applets run in web pages, but most Java programs run on servers or mobile/embedded devices.

Java programs are a major security risk.

Applet security risks are real, but in general Java is very secure.

JavaScript is a simpler version of Java.

JavaScript was named after Java for marketing reasons.

With Java, I can replace my desktop computer with a cheap “Internet appliance.”

That was an expectation 20 years ago, and it may be true today if your Android smart phone or tablet has replaced your desktop.

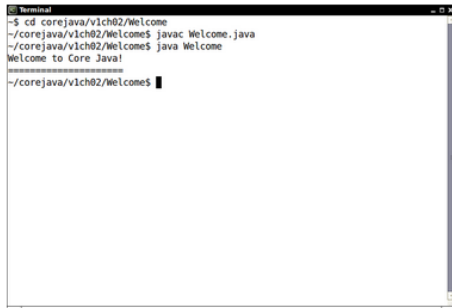
Installing Java

- Download the Java Software Development Kit (JDK) from <http://www.oracle.com/technetwork/java/javase/downloads>.
 - Navigating the Oracle site requires mastery of some Java jargon.
 - Also download and unzip the documentation of the application programming interface (API).
- Install the JDK, using the provided installer.
- Set the PATH environment variable.
- Download the sample code for these lessons.
- Install an integrated development environment (IDE) such as Eclipse, NetBeans, or IntelliJ.

Java Jargon

Java Development Kit	JDK	The software for programmers who want to write Java programs
Java Runtime Environment	JRE	The software for consumers who want to run Java programs
Standard Edition	SE	The Java platform for use on desktops and simple server applications
Enterprise Edition	EE	The Java platform for complex server applications
Micro Edition	ME	The Java platform for use on cell phones and other small devices
Update	u	Oracle's term for a bug fix release
NetBeans	—	Oracle's integrated development environment

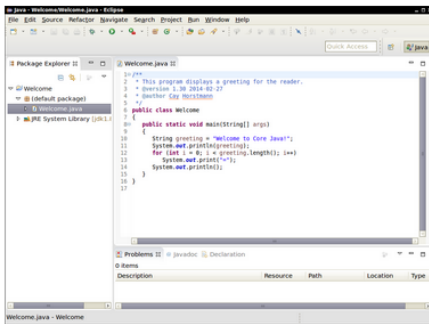
Using the Command-Line Tools



```
Terminal
~$ cd corejava/v1ch02/Welcome
~/corejava/v1ch02/Welcome$ javac Welcome.java
~/corejava/v1ch02/Welcome$ java Welcome
Welcome to Core Java!
~/corejava/v1ch02/Welcome$
```

- Open a terminal window.
- Change to the directory where you unzipped the sample code for these lessons.
- `cd corejava/v1ch02/Welcome`
- `javac Welcome.java`
- `java Welcome`
- Congratulations! You have just compiled and run your first Java program.

Using an Integrated Development Environment



- It is useful to know how to compile programs from the command line.
- For day-to-day work, integrated development environments are more convenient.
- Excellent choices are the freely available Eclipse, NetBeans, and IntelliJ IDEA.
- I use Eclipse in these lessons. Download from <http://eclipse.org>. Unzip, then launch Eclipse.
- Select File → New → Project from the menu.
- Make a Java project with existing sources at corejava/v1ch02/Welcome
- Run the program.
- Introduce an error: Change String to string
 - Note the wiggly underline.
 - Note the error in the Problems pane.

We will not use 'Hello, World!'

```
■ public class FirstSample
{
    public static void main(String[] args)
    {
        System.out.println("We will not use 'Hello, World!'");
    }
}
```

- Java is case sensitive: Main $\neq$ main.
- Keywords public, static, etc.
- Braces { } are used for blocks.
- Statements end in semicolons.
- Everything is inside a class.
- By convention, class names are CamelCase.
- Source file *must* be called FirstSample.java.

Method Calls

- `System.out.println("We will not use 'Hello, World!'");`

- General structure:

object.method(parameters)

- Parentheses needed even if there are no parameters:

`System.out.println();`

Comments

- Single-line comments:

```
// like this
```

- Multi-line comments:

```
/*  
    like  
    this  
    ...  
*/
```

- Documentation comments:

```
/**  
 * This is the first sample program in Core Java Chapter 3  
 * @version 1.01 1997-03-22  
 * @author Gary Cornell  
 */
```

Integers

- Four integer types:

int	4 bytes	-2,147,483,648 to 2,147,483, 647 (just over 2 billion)
short	2 bytes	-32,768 to 32,767
long	8 bytes	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
byte	1 byte	-128 to 127

- Literals:

- Long: 4000000000L
- Hex: 0xCAFE
- Binary: 0b1111_0100_0010_0100_0000

Floating-Point Numbers

- Two floating-point types:

float	4 bytes	Approximately $\pm 3.40282347\text{E}+38\text{F}$ (6–7 significant decimal digits)
double	8 bytes	Approximately $\pm 1.79769313486231570\text{E}+308$ (15 significant decimal digits)

- float literals: 0.5F
- Only use float if a library requires it.
- Special values `Double.POSITIVE_INFINITY`, `Double.NEGATIVE_INFINITY`, `Double.NaN`.

The char Type

- Originally used to describe Unicode characters.
- Nowadays: “Code Units” in the UTF-16 encoding.
- Every Unicode character requires one or two char values:
 - A has “code point” U+0041 and is encoded as a single char value (hex 0041 or decimal 65).
 - ☺ has “code point” U+1D546 and is encoded by two code units with hex values D835 DD46.
- Avoid char unless you *know* that you won't run into Unicode characters $\geq$ U+10000.
- Character literals enclosed in single quotes: 'A', '\n', '\u2122'.

The boolean Type

- Two values: false, true.
- No conversion between int and boolean.
- All together, Java has eight *primitive types*:
 - int, long, short, byte
 - double, float
 - char
 - boolean

Variables

- Every variable must be declared with a type, which comes before the name:

```
int vacationDays;
```

- Initialization is optional:

```
int vacationDays = 12;
```

- Declaration can appear anywhere in block.

- Declare as close as possible to first use.

- Can assign new value with = operator:

```
vacationDays = 11;
```

- Constant declared with final:

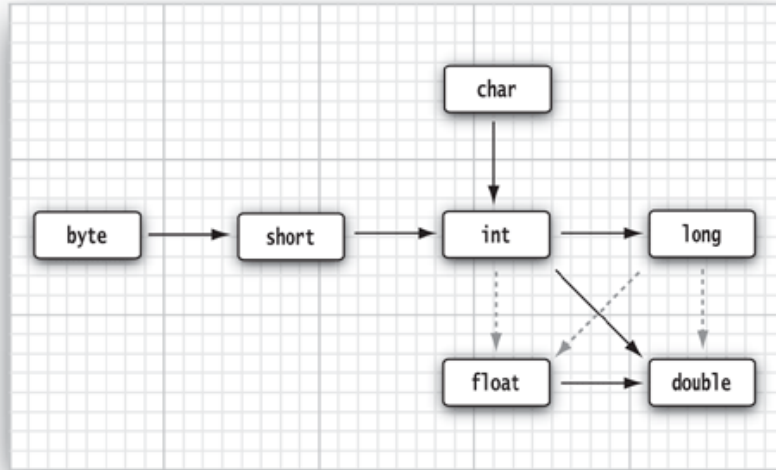
```
final int vacationDays = 12;
```

Mathematical Operators and Functions

- Arithmetic: $+$, $-$, $*$, $/$
- Integer division and modulus: $/$ and $\%$ (with integer operands):
 - $15 / 2$ is 7.
 - $15 \% 2$ is 1.
 - $15.0 / 2$ is 7.5.
- $\text{Math.sqrt}(x)$ is $\sqrt{x}$.
- $\text{Math.pow}(a, b)$ is a^b
- $\text{Math.floorMod}(a, b)$ is like $a \% b$ with better behavior for negative values:
 - $\text{Math.floorMod}(-15, 2)$ is 1.
- Trig and log functions: Math.sin , Math.log , ...

Type Conversions

- These conversions are automatic:



(Dotted arrows indicate possible precision loss.)

- Use casts for explicit conversions:

```
double x = 9.997;  
int nx = (int) x;  
int rx = (int) Math.round(x)
```

More Operators

- Combining assignment with operators:

```
n += 4; // Same as n = n + 4
```

- Also -=, *=, /=, %=, and so on.
- Increment, decrement:

```
n++; n--;
```

- Relational operators: ==, !=, <, <=, >, >=
- boolean operators: &&, ||, !
- Bitwise operators: &, |, ^, ~, >>, >>>, <<
- Conditional operator:

```
x < y ? x : y
```

Enumerated Types

- A type with a restricted set of values.
- Example:

```
enum Size { SMALL, MEDIUM, LARGE, EXTRA_LARGE };
```

- Can declare variable of this type:

```
Size s = Size.MEDIUM;
```

- Variable of type s can only hold size values or null.

Strings

- Sequences of Unicode characters.
- String literals enclosed in double quotes: "Java\u2122".
- Instances of String class.
- Use substring method to extract substrings:

```
String greeting = "Hello";  
String s = greeting.substring(0, 3);
```

- Positions start at zero.
- Last position is *excluded*—`s.substring(a, b)` has length `b - a`.
- Positions are code units (not code points).

String Concatenation

- Concatenation (+) joins strings:

```
String expletive = "Expletive";  
String PG13 = "deleted";  
String message = expletive + PG13;
```

- If one operand is not a string, it is turned into a string:

```
int age = 13;  
String rating = "PG" + age;
```

More About Strings

- Strings are immutable: `greeting.substring(0, 3)`, `greeting + "!"` don't change `greeting`.
- Can reassign string *variables*:

```
greeting = greeting.substring(0, 3) + "p!";
```

- String comparison:

```
"Hello".equals(greeting)  
"Hello".equalsIgnoreCase(greeting)
```

- **Caution:** Do *not* use the `==` operator.

```
"Hello".substring(0, 3) == "Hel" // probably false
```

- Empty string `""` has length 0.
- `null` indicates no string at all.

Code Points and Code Units

- `s.length()` is the number of code units (not Unicode characters).
- `s.charAt(i)` is the i^{th} code unit.
- To get the i^{th} code point:

```
int index = s.offsetByCodePoints(0, i);  
int cp = s.codePointAt(index);
```

- To get all code points:

```
int[] codePoints = str.codePoints().toArray();
```

The String API

- Many other useful String methods.
- `trim` yields a new string, trimming leading and trailing white space.
- `toLowerCase` yields a new string that converts all uppercase characters to lowercase.
- `indexOf`, `lastIndexOf` find the location of a substring.
- Check out the online API documentation!

Console Input

- Read console input from a Scanner:

```
Scanner in = new Scanner(System.in);
```

- Use `nextLine`, `next`, `nextInt`, `nextDouble` to read input:

```
int age = in.nextInt();
```

- Need to add import statement:

```
import java.util.*;
```

Formatted Output

- Use `printf` for formatted printing:

```
System.out.printf("Price:%8.2f", 10000.0 / 3.0); // Prints Price: 3333.33
```

- Use `f` for floating-point, `d` for integer, `s` for strings and other objects.
- Flags modify output:

```
System.out.printf("%(,.2f", -10000.0 / 3.0); // prints (3,333.33)
```

- Use `String.format` if you don't want to print:

```
String message = String.format("Hello, %s. Next year, you'll be %d", name, age);
```

File Input and Output

- Reading from a file:

```
Scanner in = new Scanner(Paths.get("myfile.txt"), "UTF-8");
```

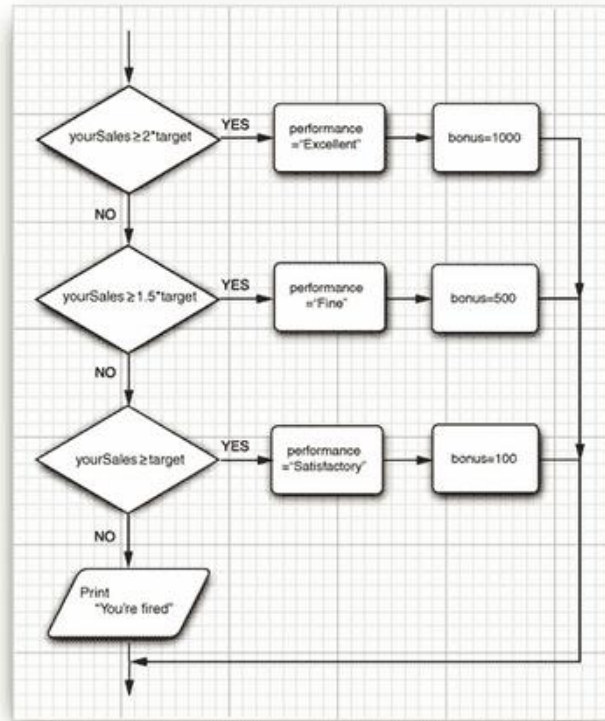
- Writing to a file:

```
PrintWriter out = new PrintWriter("myfile.txt", "UTF-8");  
out.println(...);
```

- Need to import java.io, java.nio.path packages.
- Need to tag main like this:

```
public static void main(String[] args) throws IOException  
{  
    Scanner in = new Scanner(Paths.get("myfile.txt"), "UTF-8");  
    . . .  
}
```

The if Statement

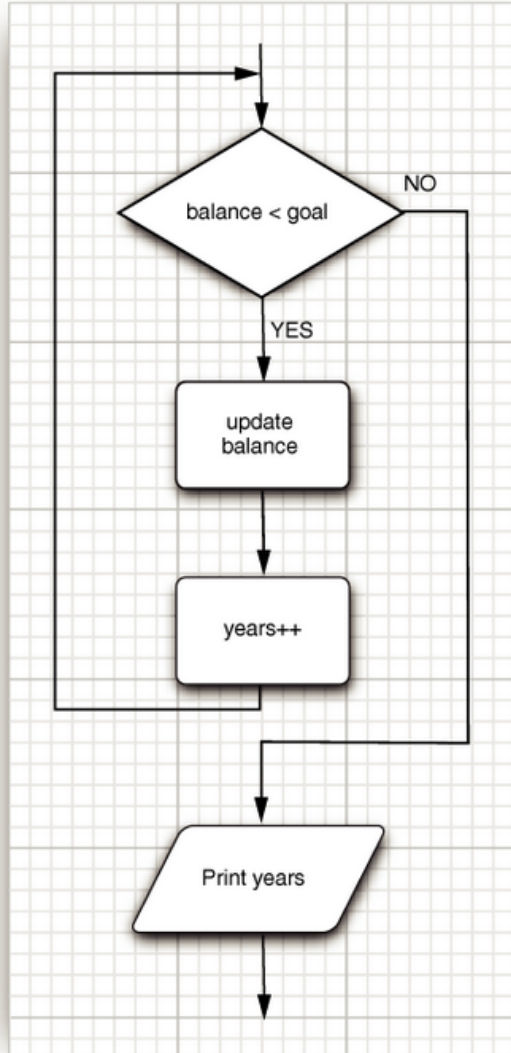


```
if (yourSales >= 2 * target)
{
    performance = "Excellent";
    bonus = 1000;
}
else if (yourSales >= 1.5 * target)
{
    performance = "Fine";
    bonus = 500;
}
else if (yourSales >= target)
{
    performance = "Satisfactory";
    bonus = 100;
}
else
{
```

```
    System.out.println("You're fired");
```

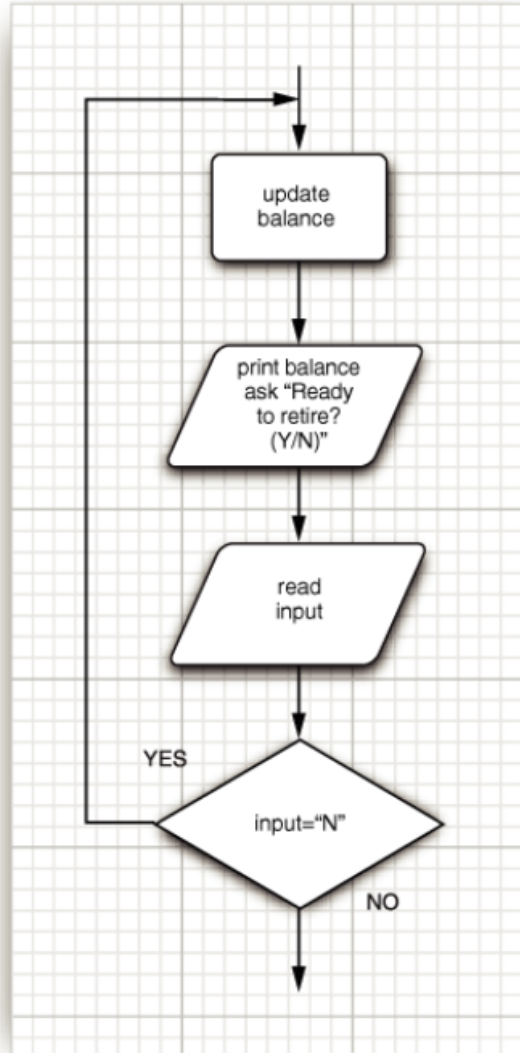
```
}
```

The while Statement



```
while (balance < goal)
{
    balance += payment;
    double interest = balance * interestRate / 100;
    balance += interest;
    years++;
}
System.out.println(years + " years.");
```

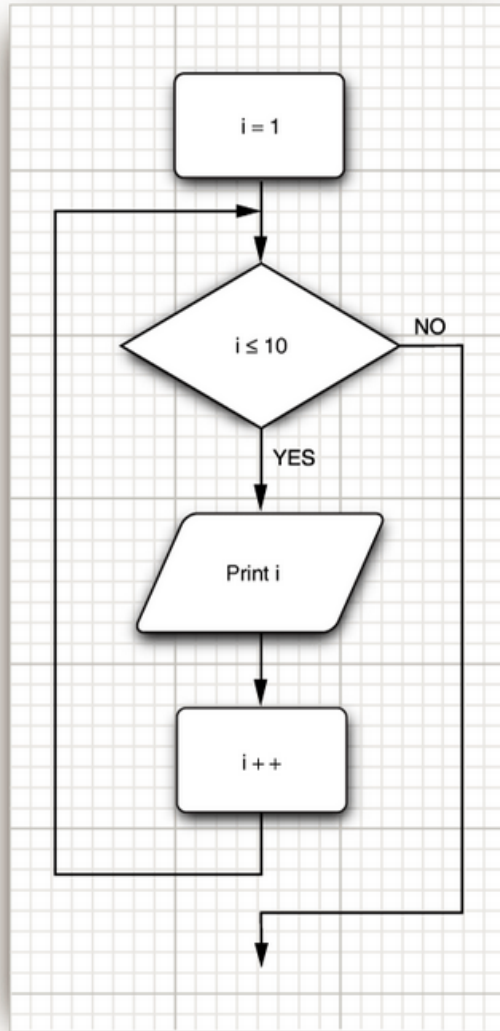
The do/while Statement



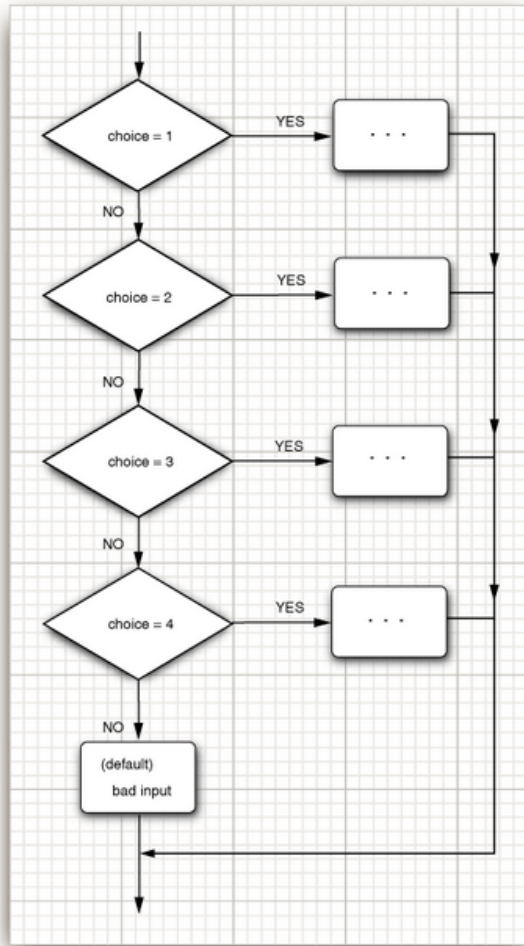
```
do
{
    balance += payment;
    double interest = balance * interestRate / 100;
    balance += interest;
    year++;
    // print current balance
    . . .
    // ask if ready to retire and get input
    . . .
}
while (input.equals("N"));
```


The for Statement

```
for (int i = 1; i <= 10; i++)  
    System.out.println(i);
```



The switch Statement



```
Scanner in = new Scanner(System.in);
System.out.print("Select an option (1, 2, 3, 4) ");
int choice = in.nextInt();
switch (choice)
{
    case 1:
        . . .
        break;
    case 2:
        . . .
        break;
    case 3:
        . . .
        break;
    case 4:
        . . .
        break;
    default:
        // bad input
        . . .
        break;
}
```

The break Statement

- Breaks out of a loop:

```
while (years <= 100)
{
    balance += payment;
    double interest = balance * interestRate / 100;
    balance += interest;
    if (balance >= goal) break;
    years++;
}
```

- Can always replace break by placing the remainder inside if and adding another termination condition:

```
while (years <= 100 && balance < goal)
{
    balance += payment;
    double interest = balance * interestRate / 100;
    balance += interest;
    if (balance < goal)
        years++;
}
```

- A “labeled break” lets you break out of multiple nested loops.

The continue Statement

- Continues with the next loop iteration:

```
Scanner in = new Scanner(System.in);
while (sum < goal)
{
    System.out.print("Enter a number: ");
    n = in.nextInt();
    if (n < 0) continue;
    sum += n; // not executed if n < 0
}
```

- If $n < 0$, then $sum < goal$ is checked next.
- Can always replace continue by placing the remainder inside if.
- A “labeled continue” lets you continue in an outer loop.

Big Numbers

- If the precision of int and double doesn't suffice, use BigInteger or BigDecimal.
- Turn an int into a BigInteger:

```
BigInteger a = BigInteger.valueOf(100);
```

- Use methods such as add and multiply to combine big numbers:

```
BigInteger c = a.add(b); // c = a + b  
BigInteger d = c.multiply(b.add(BigInteger.valueOf(2))); // d = c * (b + 2)
```

- Example: Pick 60 numbers out of 490. Your odds of winning are 1 in
716395843461995557415116222540092933411717612789263493493351013459481104668848. Good luck!

Arrays

- `int[]` is an array of integers.
- Array variable declaration:

```
int[] a;
```

- `new` operator creates array:

```
int[] a = new int[100];
```

- Array indexes are from 0 to `a.length - 1`.
- Use `[]` to access elements:

```
for (int i = 0; i < a.length; i++)  
    System.out.println(a[i]);
```

- Or use the “for each” loop:

```
for (int element : a)  
    System.out.println(element);
```

Supplying Element Values

- Array initializers:

```
int[] smallPrimes = { 2, 3, 5, 7, 11, 13 };
```

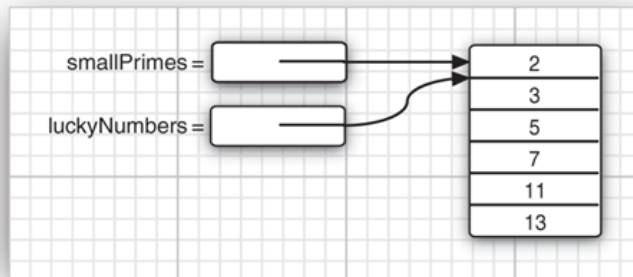
- Anonymous arrays:

```
new int[] { 17, 19, 23, 29, 31, 37 }
```

Copying Arrays

- Copying array variables yields two references to the same array:

```
int[] luckyNumbers = smallPrimes;  
luckyNumbers[5] = 12; // now smallPrimes[5] is also 12
```



- Use `Arrays.copyOf` to make a true copy:

```
int[] copiedLuckyNumbers = Arrays.copyOf(luckyNumbers, luckyNumbers.length);
```

Multidimensional Arrays

- `int[][]` is an array of arrays or a two-dimensional array:

```
int[][] magicSquare =  
{  
    {16, 3, 2, 13},  
    {5, 10, 11, 8},  
    {9, 6, 7, 12},  
    {4, 15, 14, 1}  
};
```

- Without initializer:

```
int[][] magicSquare = new int[ROWS][COLUMNS];
```

- Use two indexes to access element: `magicSquare[1][2]` is 11.
- Use this loop to traverse the elements:

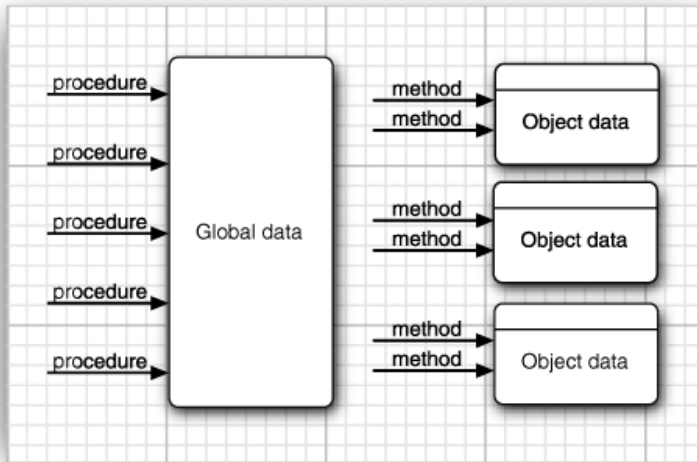
```
for (int[] row : magicSquare)  
    for (int element : row)  
        do something with value
```

- If the rows have different lengths, the array is “ragged”:

```
int[][] triangle = new int[ROWS][];  
for (int i = 0; i < ROWS; i++) triangle[i] = new int[i + 1];
```


Why OOP?

- 1970s: “Structured” programming.
 - Algorithms + Data Structures = Programs.
 - Procedures operate on shared data.
- 1980s: Object-oriented programming.
 - Each object has data and methods.
 - Encapsulation: Only methods can access object data.



- Java is thoroughly object-oriented.
 - Everything other than a primitive type value is an object.

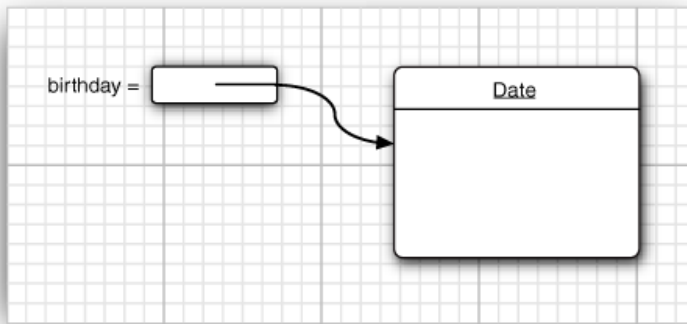
Classes and Objects

- Class: “Cookie cutter” for objects.
- Describes object data and method behavior.
- Object = *instance* of class.
- Object has:
 - Behavior
 - State
 - Identity
- OOP starts with identifying classes:
 - Nouns are often classes: Item, Order, and so on.
 - Verbs are often methods: add an item to an order, ship an order.

Using Predefined Classes

- Math is an atypical class. You don't make objects to call `Math.sqrt(x)`.
- The Date class is more typical. Each object describes a point in time such as “December 31, 1999, 23:59:59 GMT”
- *Construct* instance with new operator:

```
Date birthday = new Date();
```



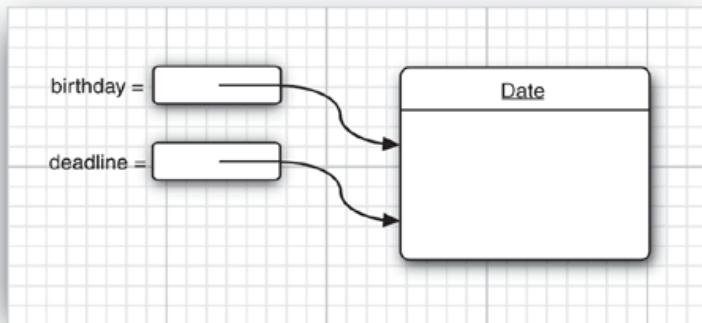
- Now you can call methods:

```
String s = birthday.toString();
```

Object Variables

- An object variable holds a *reference* to an object.
- Copying a variable makes a copy of the reference:

```
deadline = birthday;
```



- A null reference refers to no object:

```
deadline = null;
```

- **Caution:** Don't call a method on null.

```
if (deadline != null)
    s = deadline.toString();
```

Working with LocalDate

- A Date is a point in time, measured in UTC.
- A LocalDate is a date (day, month, year) in a particular location.
- Use *factory methods* to create instances:

```
LocalDate rightNow = LocalDate.now();  
LocalDate newYearsEve = LocalDate.of(1999, 12, 31);
```

- LocalDate methods:

```
LocalDate aThousandDaysLater = newYearsEve.plusDays(1000);  
year = aThousandDaysLater.getYear(); // 2002  
month = aThousandDaysLater.getMonthValue(); // 09  
day = aThousandDaysLater.getDayOfMonth(); // 26
```

- How is a LocalDate stored? How do these methods do their job? You don't know, and you don't care. That's encapsulation.

Accessor and Mutator Methods

- Accessor method doesn't modify object state.
- All `LocalDate` methods are accessors.
- Older version of calendar date class has mutator methods:

```
GregorianCalendar someDay = new GregorianCalendar(1999, 11, 31);
someDay.add(Calendar.DAY_OF_MONTH, 1000);
// someDay has been mutated
int year = someDay.get(Calendar.YEAR); // 2002
```

- **Tip:** Minimize mutator methods. They make it more difficult to share objects in concurrent programs.

Defining Your Own Classes

```
class Employee
{
    // Fields
    private String name;
    private double salary;
    private LocalDate hireDay;

    // Constructors
    public Employee(String n, double s, int year, int month, int day)
    {
        name = n;
        salary = s;
        hireDay = LocalDate.of(year, month, day);
    }

    // Methods
    public String getName() { return name; }
    . . .
}
```

Constructors

- In general, instance fields are private.
- Initialized in constructor:

```
public Employee(String n, double s, int year, int month, int day)
{
    name = n;
    salary = s;
    hireDay = LocalDate.of(year, month, day);
}
```

- The call

```
new Employee("James Bond", 100000, 1950, 1, 1)
```

sets the fields as follows:

```
name = "James Bond";
salary = 100000;
hireDay = LocalDate.of(1950, 1, 1);
```

- Name of constructor = class name.
- Constructor only works with new.

Implicit and Explicit Parameters

- Methods access and modify fields:

```
public void raiseSalary(double byPercent)
{
    double raise = salary * byPercent / 100;
    salary += raise;
}
```

- In the call `number007.raiseSalary(5)`, these steps occur:

```
double raise = number007.salary * 5 / 100;
number007.salary += raise;
```

- The call depends on two parameters:

- `byPercent` is an *explicit* parameter.
- The object on which the method is invoked is the *implicit* parameter.

- Can optionally use `this` to denote implicit parameter:

```
public void raiseSalary(double byPercent)
{
    double raise = this.salary * byPercent / 100;
    this.salary += raise;
}
```

Benefits of Encapsulation

- Note the private field and public method:

```
private String name;  
  
public String getName() { return name; }
```

- Benefit 1: The field is “read-only”.
- Benefit 2: The internal representation can evolve:

```
private String firstName;  
private String lastName;  
  
public string getName() { return firstName + " " + lastName; }
```

Final and Static Fields

- A final field cannot change:

```
private final String name;
```

- **Caution:** A final object can still be mutated:

```
private final StringBuilder evaluations;  
public Employee() { evaluations = new StringBuilder(); . . . }  
public void giveGoldStar() { evaluations.append("Gold star!\n"); }
```

- A static field exists once per class:

```
private static int nextId; // one field per class  
private int id; // one field per object  
public void setId() { id = nextId; nextId++; }
```

- A static final field is a shared constant:

```
public class Math  
{  
    public static final double PI = 3.14159265358979323846;  
    // Accessible anywhere as Math.PI  
    . . .  
}
```

Static Methods

- A static method doesn't operate on objects.
- Example: `Math.pow(a, b)` computes a^b without using a `Math` object.
- In other words, a static method uses no `this`.
- Static methods can only access static fields:

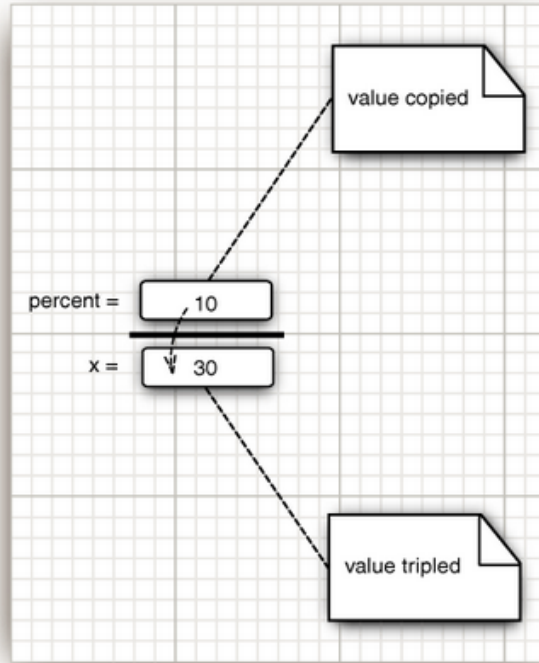
```
public static int getNextId()
{
    return nextId; // returns static field
}
```

- Supply class name when calling the method:

```
int n = Employee.getNextId();
```

- The `main` method is static because no objects have been constructed when the program starts.

Call by Value



- Call by value: The method gets copies of the argument values.
- A method cannot change the contents of *variables* passed to it.
- Example:

```
public static void tripleValue(double x) // doesn't work
{
    x = 3 * x;
}
```

- In the following call, the `percent` variable is *not* changed:

```
double percent = 10;
tripleValue(percent);
```

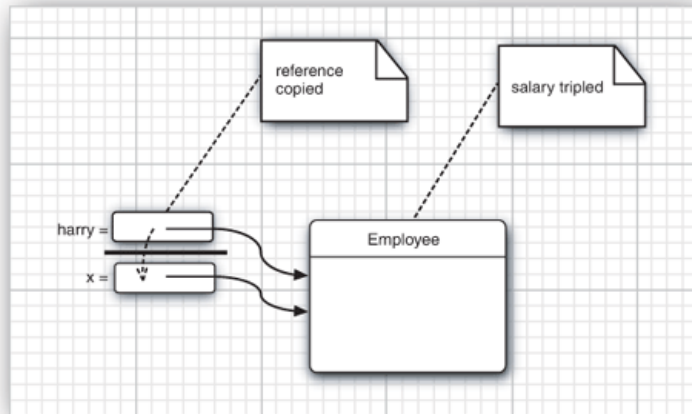
Call with Object References

- A method can mutate objects:

```
public static void tripleSalary(Employee x) // works
{
    x.raiseSalary(200);
}
```

- In the following call, the salary is changed:

```
harry = new Employee(. . .);
tripleSalary(harry);
```



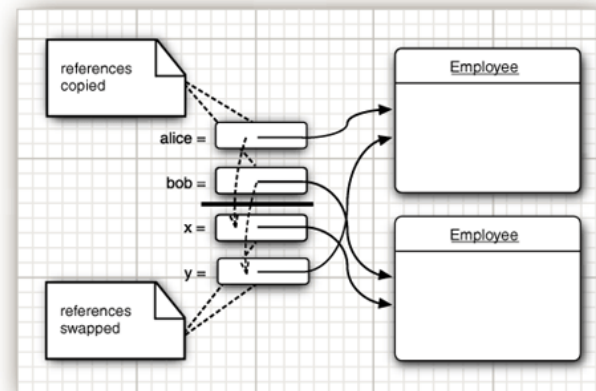
Object References Are Passed by Value

- Some people say: “In Java, numbers are passed by value and objects are passed by reference.”
- That's nonsense. In Java, everything is passed by value.
- Object *references* are passed by value.
- If objects were passed by reference, you could swap them:

```
public static void swap(Employee x, Employee y) // doesn't work
{
    Employee temp = x;
    x = y;
    y = temp;
}
```

- But in the following call, a and b are *not* swapped:

```
Employee alice = new Employee("Alice", . . .);
Employee bob = new Employee("Bob", . . .);
swap(alice, bob);
```



Overloading

- A class can have more than one constructor:

```
StringBuilder messages = new StringBuilder();  
StringBuilder todoList = new StringBuilder("To do:\n");
```

- The constructor name is *overloaded*.
- Name + parameter types = Method *signature*.
- Overloading resolution: The compiler picks the appropriate version from the argument types.
- You can overload any method:

```
String.indexOf(int)  
String.indexOf(int, int)  
String.indexOf(String)  
String.indexOf(String, int)
```

- The return type is not a part of the method signature.

Default Construction

- A field that isn't explicitly set in a constructor is 0, false, or null.
- **Caution:** Accidentally uninitialized variables can lead to null pointer errors.

```
public Employee() { name = ""; }  
.  
.  
.  
LocalDate h = harry.getHireDay();  
int year = h.getYear();
```

- If a class has *no constructor*, a no-argument constructor is provided.
- It sets all fields to their default values.
- If a class has at least one constructor, the no-argument constructor is not provided.
- But you can provide it:

```
public Employee() {}
```

Field Initialization

- You can override the 0/false/null default for fields:

```
class Employee
{
    private String name = "";
    . . .
}
```

- The initialization value can be computed:

```
class Employee
{
    private static int nextId;
    private int id = assignId();
    . . .
    private static int assignId()
    {
        int r = nextId;
        nextId++;
        return r;
    }
    . . .
}
```

Construction Parameter Names

- Some programmers like single-letter parameter names:

```
public Employee(String n, double s)
{
    name = n;
    salary = s;
}
```

- For better documentation, can use a prefix:

```
public Employee(String aName, double aSalary)
{
    name = aName;
    salary = aSalary;
}
```

- Can use this to distinguish between parameters and fields:

```
public Employee(String name, double salary)
{
    this.name = name;
    this.salary = salary;
}
```

Calling Another Constructor

- A constructor can call another constructor in the *first statement*.
- Use `this` (and not the class name) for the call:

```
public Employee(double s)
{
    // calls Employee(String, double)
    this("Employee #" + nextId, s);
    nextId++;
}
```

- Allows you to factor out common construction code.
- Keyword reuse: Not related to using `this` for the implicit parameter.

Initialization Blocks

- Class declarations can contain arbitrary blocks of code.
- Executed whenever an object is constructed:

```
class Employee
{
    private static int nextId;
    private int id;
    // object initialization block
    {
        id = nextId;
        nextId++;
    }

    public Employee(. . .) { . . . } // constructor
    . . .
}
```

- Static initialization block is executed when class is loaded:

```
static
{
    Random generator = new Random();
    nextId = generator.nextInt(10000);
}
```

Packages

- Related classes are organized into packages:

```
java.lang  
java.util  
java.time  
... 
```

- Avoids name conflict: `java.util.Date` $\neq$ `java.sql.Date`
- Use reverse domain name for your own packages: `com.horstmann.corejava`
- Packages do not nest: No relationship between `java.util` and `java.util.jar`

Imports

- Can access classes from any package with fully qualified name:

```
java.time.LocalDate today = java.time.LocalDate.now();
```

- Import statements remove the tedious repetition:

```
import java.time.*;  
.  
.  
.  
LocalDate today = LocalDate.now();
```

- Can import single class:

```
import java.time.LocalDate;
```

- Cannot have multiple wildcards (import java.*.*).
- If two packages import the same class, you still need fully qualified names:

```
import java.util.*;  
import java.sql.*;  
.  
.  
.  
Date today; // Error--java.util.Date or java.sql.Date?
```

- Or add a disambiguating import to the wildcard imports:

```
import java.util.Date;
```

Static Imports

- Imports static fields and methods:

```
import static java.lang.System.*;
```

- Now you can refer to `System.out` and `System.exit` without the class name:

```
out.println("Goodbye, World!"); // i.e., System.out  
exit(0); // i.e., System.exit
```

- Can import a specific method or field:

```
import static java.lang.System.out;
```

- Can be handy for mathematical functions:

```
import static java.lang.Math.*;  
.  
.  
.  
r = sqrt(pow(x, 2) + pow(y, 2));
```


Adding a Class to a Package

- Put a package declaration at the top of the file:

```
package com.horstmann.corejava;  
  
public class Employee  
{  
    . . .  
}
```

- A class without a package declaration is in the *default package*.
- Place the source file into a subdirectory that matches the package name.

```
. (base directory)  
├── PackageTest.java  
├── PackageTest.class  
└── com/  
    └── horstmann/  
        └── corejava/  
            ├── Employee.java  
            └── Employee.class
```

- Compile from the base directory:

```
javac com/horstmann/corejava/Employee.java
```

Package Scope

- A feature that is not public, private, or protected (see next lesson) has *package scope*.
- All methods of all classes of a package can access it.
- **Caution:** Package scope is open-ended. Any class can add itself to (almost) any package.
- This Window class is in the `java.awt` package:

```
public class Window extends Container
{
    String warningString;
    . . .
}
```

- In Java $\leq$ 1.1, I was able to add my own class like this:

```
package java.awt;
...
window.warningString = "Trust me!";
```

- No longer possible—`java` and `javax` packages enjoy special protection.
- You can *seal* packages in a JAR file.

The Class Path

- Class path=list of directories and JAR files.
- JAR file=zip file containing class files.
- Directories are *base* directories, containing package directories (such as com/horstmann/corejava).
- Class path elements are separated by : (Unix) or ; (Windows).
- Can include current directory as .
- Can specify *all* JAR files in a directory as *directory/ **
 - **Caution:** In Unix, must escape * as '*' or *
- Pass to javac/java with -classpath option:

```
java -classpath /home/user/classdir:../home/user/archives/archive.jar MyProg
```

- Or set CLASSPATH environment variable:

```
export CLASSPATH=/home/user/classdir:../home/user/archives/archive.jar
```

Class Design Hints

- Always keep data private.
- Always initialize data.
- Don't use too many basic types in a class:

```
public class Customer
{
    private String street;
    private String city;
    private String state;
    private int zip;
    ...
}
```

⇒

```
public class Customer
{
    private Address shippingAddress;
    ...
}
```

- Not all fields need field accessors and mutators:

```
public class Address
{
    ...
    private int zip;
    public void setZip(int newZip) { zip = newZip; }
    ...
}
```

Class Design Hints

- Break up classes that have too many responsibilities:

```
public class CardDeck // bad design
{
    private int[] value;
    private int[] suit;
    ...
    public void shuffle() { . . . }
    public int getTopValue() { . . . }
    public int getTopSuit() { . . . }
    public void draw() { . . . }
}
```

⇒

```
public class CardDeck
{
    private Card[] cards;
    ...
    public void shuffle() { . . . }
    public Card getTop() { . . . }
    public void draw() { . . . }
}

public class Card
{
    private int value;
    private int suit;
    ...
    public int getValue() { . . . }
    public int getSuit() { . . . }
}
```

- Make the names of your classes and methods reflect their responsibilities.
- Prefer immutable classes.